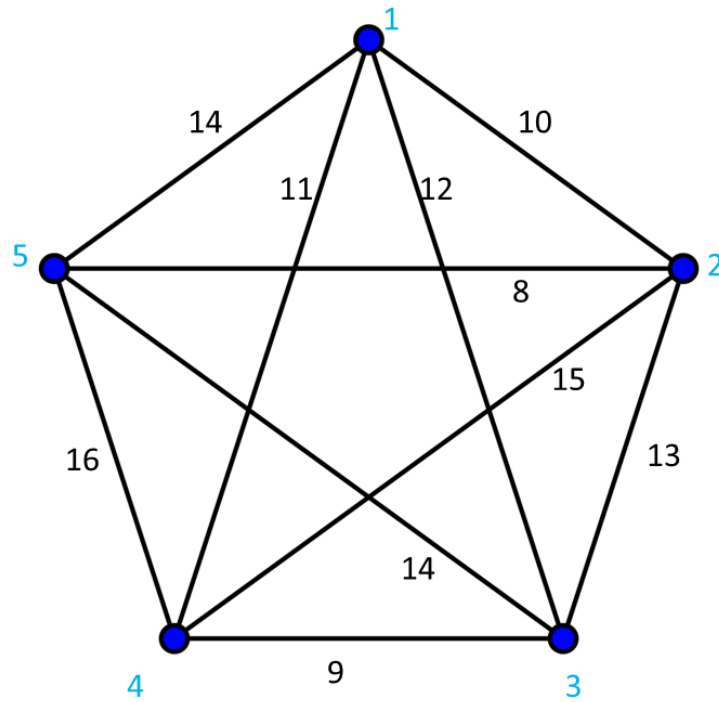




Algorithm 1: The ACO Metaheuristic

```

1 Set parameters, initialize pheromone trails;
2 while termination condition is not met do
3   ConstructAntSolutions;
4   ApplyLocalSearch (optional);
5   UpdatePheromones;
  
```



Algorithm 1: The ACO Metaheuristic

```

1 Set parameters, initialize pheromone trails;
2 while termination condition is not met do
3   for  $i \leftarrow 1$  to  $N$  do
4     for  $j \leftarrow 1$  to  $S$  do
5       By applying a stochastic state transition rule to choose the next
        city to move;
6   UpdatePheromones;
  
```

STEP 1: INITIALISATION

Get distance matrix

In [1]:



```
### Distance matrix

num_cities = 5
num_ants = 3

d = [[ 0, 10, 12, 11, 14],
      [10, 0, 13, 15, 8],
      [12, 13, 0, 9, 14],
      [11, 15, 9, 0, 16],
      [14, 8, 14, 16, 0]];
```

Compute Heuristic Matrix

In [2]:



```
### Heuristic matrix

# eta_{ij} = 1/d_{ij}

H = [[ round(1/d[i][j],4) if i != j else 0 for j in range(5)] for i in range(5)]

print('\n'.join([''.join(['{:10}'.format(item) for item in row]) for row in H]))
```

0	0.1	0.0833	0.0909	0.0714
0.1	0	0.0769	0.0667	0.125
0.0833	0.0769	0	0.1111	0.0714
0.0909	0.0667	0.1111	0	0.0625
0.0714	0.125	0.0714	0.0625	0

Initialise Pheromone Matrix

In [3]:



```
### Pheromone matrix

# tau_{ij} at t=0

T = [[ 1 for j in range(5)] for i in range(5)]

print('\n'.join([''.join(['{:10}'.format(item) for item in row]) for row in T]))
```

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

LOOP iteration t =1

STEP 3: CONSTRUCT ANT SOLUTIONS

Put ants on the graph: we put them all on node 1

For each ant build a solution:

For Ant 1:

- init: all nodes are in the allowed nodes set

At each city:

- remove current node from allowed nodes
- compute transition probabilities to allowed nodes
- select next node according to the computed probabilities

CITY 1

ANT1: remove current node from allowed nodes

In [4]:

```
# eta{1j} = 0 (set col 1 to all zeros to exclude current city)
H = [[ round(1/d[i][j],4) if i != j and j != 0 else 0 for j in range(5)] for i in range(5)]
print('\n'.join([''.join(['{:10}'.format(item) for item in row]) for row in H]))
```

0	0.1	0.0833	0.0909	0.0714
0	0	0.0769	0.0667	0.125
0	0.0769	0	0.1111	0.0714
0	0.0667	0.1111	0	0.0625
0	0.125	0.0714	0.0625	0

Formula Transition Probabilities

The probability for ant k to go from city i to city j while in its t -th tour is:

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}(t)]^\beta}{\sum_{h \in J_i^k} [\tau_{ih}(t)]^\alpha \cdot [\tau_{ih}(t)]^\beta} & \text{if } j \text{ has not been visited,} \\ 0 & \text{otherwise} \end{cases}$$

ANT1: compute transition probabilities to allowed nodes

- $k = 1$ (ant 1)
- $i = 1$ (curr node 1)
- $t = 1$ (iteration 1)
- $\alpha = 1$ (fixed parameter)
- $\beta = 2$ (fixed parameter)

In [5]:



```
# compute numerators: P11, P12, P13, P14, P15
```

```
#Numerators
```

```
N11 = 1**1 * 0**2
```

```
N12 = 1**1 * 0.1**2
```

```
N13 = 1**1 * 0.0833**2
```

```
N14 = 1**1 * 0.0909**2
```

```
N15 = 1**1 * 0.0714**2
```

```
#Denominator
```

```
SUM = N11+N12+N13+N14+N15
```

```
#Probabilities
```

```
P11 = N11/SUM
```

```
P12 = N12/SUM
```

```
P13 = N13/SUM
```

```
P14 = N14/SUM
```

```
P15 = N15/SUM
```

```
#####
```

```
print("N11: ", N11)
```

```
print("N12: ", N12)
```

```
print("N13: ", N13)
```

```
print("N14: ", N14)
```

```
print("N15: ", N15)
```

```
print("\nSUM: ", SUM, "\n")
```

```
print("P11: ", N11/SUM)
```

```
print("P12: ", N12/SUM)
```

```
print("P13: ", N13/SUM)
```

```
print("P14: ", N14/SUM)
```

```
print("P15: ", N15/SUM)
```

```
N11: 0
```

```
N12: 0.010000000000000002
```

```
N13: 0.00693889
```

```
N14: 0.008262809999999999
```

```
N15: 0.00509796
```

```
SUM: 0.03029966
```

```
P11: 0.0
```

```
P12: 0.3300367066825173
```

```
P13: 0.2290088403632252
```

```
P14: 0.272703060034337
```

```
P15: 0.16825139291992056
```

ANT1: select next node according to the computed probabilities

In [6]:



```
# compute cumulative probabilities

CP11 = P11
CP12 = CP11 + P12
CP13 = CP12 + P13
CP14 = CP13 + P14
CP15 = CP14 + P15

print("CP11: ", CP11)
print("CP12: ", CP12)
print("CP13: ", CP13)
print("CP14: ", CP14)
print("CP15: ", CP15)

#pick random number rand in [0,1]

# return the city whose cumulative probability is the closest higher or equal to rand

# say rand = 0.6841

# so we get city 4

# partially constructed solution by ant 1 is now: 1 --> 4
```

```
CP11: 0.0
CP12: 0.3300367066825173
CP13: 0.5590455470457425
CP14: 0.8317486070800795
CP15: 1.0
```

CITY 2

Next step (as above):

- remove current node from allowed nodes
- compute transition probabilities to allowed nodes
- select next node according to the computed probabilities

ANT1: remove current node from allowed nodes

In [7]:



```
# eta{4j} = 0 (set col 4 to all zeros to exclude current city)
H = [[ round(1/d[i][j],4) if i != j and j != 0 and j != 3 else 0 for j in range(5)] for i in range(5)]
print('\n'.join([''.join(['{:10}'.format(item) for item in row]) for row in H]))
```

0	0.1	0.0833	0	0.0714
0	0	0.0769	0	0.125
0	0.0769	0	0	0.0714
0	0.0667	0.1111	0	0.0625
0	0.125	0.0714	0	0

ANT1: compute transition probabilities

- $k = 1$ (ant 1)
- $i = 1$ (curr node 4)
- $t = 1$ (iteration 1)
- $\alpha = 1$ (fixed parameter)
- $\beta = 2$ (fixed parameter)

In [8]:



```
# compute numerators: P11, P12, P13, P14, P15
```

```
#Numerators
```

```
N41 = 1**1 * 0**2
```

```
N42 = 1**1 * 0.0667**2
```

```
N43 = 1**1 * 0.1111**2
```

```
N44 = 1**1 * 0**2
```

```
N45 = 1**1 * 0.0625**2
```

```
#Denominator
```

```
SUM = N41+N42+N43+N44+N45
```

```
#Probabilities
```

```
P41 = N41/SUM
```

```
P42 = N42/SUM
```

```
P43 = N43/SUM
```

```
P44 = N44/SUM
```

```
P45 = N45/SUM
```

```
#####
```

```
print("N11: ", N41)
```

```
print("N12: ", N42)
```

```
print("N13: ", N43)
```

```
print("N14: ", N44)
```

```
print("N15: ", N45)
```

```
print("\nSUM: ", SUM, "\n")
```

```
print("P11: ", N41/SUM)
```

```
print("P12: ", N42/SUM)
```

```
print("P13: ", N43/SUM)
```

```
print("P14: ", N44/SUM)
```

```
print("P15: ", N45/SUM)
```

```
N11: 0
```

```
N12: 0.00444889
```

```
N13: 0.01234321
```

```
N14: 0
```

```
N15: 0.00390625
```

```
SUM: 0.02069835
```

```
P11: 0.0
```

```
P12: 0.2149393550693654
```

```
P13: 0.5963378723424814
```

```
P14: 0.0
```

```
P15: 0.18872277258815315
```

ANT1: select next node according to the computed probabilities

In [9]:



```
# compute cumulative probabilities

CP41 = P41
CP42 = CP41 + P42
CP43 = CP42 + P43
CP44 = CP43 + P44
CP45 = CP44 + P45

print("CP41: ", CP41)
print("CP42: ", CP42)
print("CP43: ", CP43)
print("CP44: ", CP44)
print("CP45: ", CP45)

#pick random number rand in [0,1]

# return the city whose cumulative probability is the closest higher or equal to rand

# say rand = 0.4024

# so we get city 3

# partially constructed solution by ant 1 is now: 1 --> 4 --> 3
```

```
CP41: 0.0
CP42: 0.2149393550693654
CP43: 0.8112772274118467
CP44: 0.8112772274118467
CP45: 0.9999999999999999
```

Repeat this process (further steps and further ants) and we get:

Ant1: 1 --> 4 --> 3 --> 5 --> 2 --> 1 (length: 52)

Ant2: 1 --> 4 --> 2 --> 5 --> 3 --> 1 (length: 60)

Ant3: 1 --> 4 --> 5 --> 2 --> 3 --> 1 (length: 60)

STEP 5: UPDATE PHEROMONE

Pheromone Update Formula

$$\Delta_{ij}^k(t) = \begin{cases} Q/L_k & \text{if ant } k \text{ used edge } (i, j) \text{ in its tour,} \\ 0 & \text{otherwise} \end{cases}$$

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \sum_{k=1}^S \Delta \tau_{ij}^k(t)$$

we fix parameters:

- evaporation rate $\rho = 0.5$
- $Q = 1$

Evaporation

In [10]:



```
### evaporation: (1 - rho) * T (pheromone matrix)

rho = 0.5

# pheromone after evapoation
T = [[ (1-rho)*T[i][j] for j in range(5)] for i in range(5)]

print('\n'.join([''.join(['{:10}'.format(item) for item in row]) for row in T]))
```

```
0.5      0.5      0.5      0.5      0.5
0.5      0.5      0.5      0.5      0.5
0.5      0.5      0.5      0.5      0.5
0.5      0.5      0.5      0.5      0.5
0.5      0.5      0.5      0.5      0.5
```

Ant1 deposit

In [11]:



```

### Ant1 deposits pheromone

# Ant1: 1 --> 4 --> 3 --> 5 --> 2 --> 1 (length: 52)

# amount deposited
delta1 = 1/52

T[0][3] += delta1 ## deposit on 1 --> 4
T[3][2] += delta1 ## deposit on 4 --> 3
T[2][4] += delta1 ## deposit on 3 --> 5
T[4][1] += delta1 ## deposit on 5 --> 2
T[1][0] += delta1 ## deposit on 2 --> 1

# pheromone after contribution of Ant1
print('\n'.join([''.join(['{:20}'.format(item) for item in row]) for row in T]))

```

```

          0.5          0.5          0.5  0.519230769230
7693          0.5
  0.5192307692307693          0.5          0.5
0.5          0.5
          0.5          0.5          0.5
0.5  0.5192307692307693
          0.5          0.5  0.5192307692307693
0.5          0.5
          0.5  0.5192307692307693          0.5
0.5          0.5

```

Ant2 deposit

In [12]:



```

### Ant2 deposits pheromone

# Ant2: 1 --> 4 --> 2 --> 5 --> 3 --> 1 (length: 60)

# amount deposited
delta2 = 1/60

T[0][3] += delta2 ## deposit on 1 --> 4
T[3][1] += delta2 ## deposit on 4 --> 2
T[1][4] += delta2 ## deposit on 2 --> 5
T[4][2] += delta2 ## deposit on 5 --> 3
T[2][0] += delta2 ## deposit on 3 --> 1

# pheromone after contribution of Ant2
print('\n'.join([''.join(['{:20}'.format(item) for item in row]) for row in T]))

```

```

          0.5          0.5          0.5  0.53589743589
7436          0.5
  0.5192307692307693          0.5          0.5
0.5  0.5166666666666667
  0.5166666666666667          0.5          0.5
0.5  0.5192307692307693
          0.5  0.5166666666666667  0.5192307692307693
0.5          0.5
          0.5  0.5192307692307693  0.5166666666666667
0.5          0.5

```

Ant3 deposit

In [13]:



```

### Ant3 deposits pheromone

# Ant3: 1 --> 4 --> 5 --> 2 --> 3 --> 1 (length: 60)

# amount deposited
delta3 = 1/60

T[0][3] += delta3 ## deposit on 1 --> 4
T[3][4] += delta3 ## deposit on 4 --> 5
T[4][1] += delta3 ## deposit on 5 --> 2
T[1][2] += delta3 ## deposit on 2 --> 3
T[2][0] += delta3 ## deposit on 3 --> 1

# pheromone after contribution of Ant3
print('\n'.join([''.join(['{:20}'.format(item) for item in row]) for row in T]))

```

```

0.5 0.5 0.5 0.552564102564
1027 0.5
0.5192307692307693 0.5 0.5166666666666667
0.5 0.5166666666666667
0.5333333333333334 0.5 0.5
0.5 0.5192307692307693
0.5 0.5166666666666667 0.5192307692307693
0.5 0.5166666666666667
0.5 0.535897435897436 0.5166666666666667
0.5 0.5

```

LOOP iteration t = 2

...

In []:

